

Module - 17:

DevSecOps

- Amjad Hossain
25th April, 2026

What we do in daily life

- Code writing and just making it work
- Just formality of “code review”
- Pipeline build was success and deployed to Dev/Staging environment
- QA perform business cases test
- Maybe few bug fixes and test again
- Release a plan
- Deploy to production
- Sanity check
- Maybe observation for hours
- Forget the past and repeat

1. Secure Coding Practices (what we do)

- **We trust user input too early**

Input is used directly in queries, file paths, or business logic without strict validation or sanitization.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.
- **We log sensitive information for debugging**
Passwords, tokens, or personal data end up in logs without masking or filtering.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.
- **We log sensitive information for debugging**
Passwords, tokens, or personal data end up in logs without masking or filtering.
- **We hardcode secrets in code or configs**
Credentials, API keys, and tokens are embedded for convenience during development and never removed.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.
- **We log sensitive information for debugging**
Passwords, tokens, or personal data end up in logs without masking or filtering.
- **We hardcode secrets in code or configs**
Credentials, API keys, and tokens are embedded for convenience during development and never removed.
- **We skip proper authentication and authorization checks in code**
Access control is assumed to be handled “somewhere else” (e.g., gateway), but not enforced at service level.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.
- **We log sensitive information for debugging**
Passwords, tokens, or personal data end up in logs without masking or filtering.
- **We hardcode secrets in code or configs**
Credentials, API keys, and tokens are embedded for convenience during development and never removed.
- **We skip proper authentication and authorization checks in code**
Access control is assumed to be handled “somewhere else” (e.g., gateway), but not enforced at service level.
- **We use outdated or unsafe libraries without verification**
Dependencies are added for functionality, not evaluated for security posture.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.
- **We log sensitive information for debugging**
Passwords, tokens, or personal data end up in logs without masking or filtering.
- **We hardcode secrets in code or configs**
Credentials, API keys, and tokens are embedded for convenience during development and never removed.
- **We skip proper authentication and authorization checks in code**
Access control is assumed to be handled “somewhere else” (e.g., gateway), but not enforced at service level.
- **We use outdated or unsafe libraries without verification**
Dependencies are added for functionality, not evaluated for security posture.
- **We don’t enforce consistent error handling**
Raw exceptions and stack traces are exposed, leaking internal implementation details.

1. Secure Coding Practices (what we do)

- **We trust user input too early**
Input is used directly in queries, file paths, or business logic without strict validation or sanitization.
- **We build queries and commands dynamically**
String concatenation is used for SQL, shell commands, or URLs instead of safe, parameterized approaches.
- **We expose more data than necessary in APIs**
Responses often return full objects (including sensitive fields) instead of carefully curated DTOs.
- **We rely on “happy path” logic only**
Edge cases, invalid states, and malicious inputs are not handled explicitly.
- **We log sensitive information for debugging**
Passwords, tokens, or personal data end up in logs without masking or filtering.
- **We hardcode secrets in code or configs**
Credentials, API keys, and tokens are embedded for convenience during development and never removed.
- **We skip proper authentication and authorization checks in code**
Access control is assumed to be handled “somewhere else” (e.g., gateway), but not enforced at service level.
- **We use outdated or unsafe libraries without verification**
Dependencies are added for functionality, not evaluated for security posture.
- **We don’t enforce consistent error handling**
Raw exceptions and stack traces are exposed, leaking internal implementation details.
- **We assume internal APIs are safe**
Internal endpoints are built with weaker validation and security assumptions than public ones.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.
- **Sensitive data in logs → Implement secure logging**
Mask/redact secrets and PII; use structured logging with filters; segregate debug vs prod logging.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.
- **Sensitive data in logs → Implement secure logging**
Mask/redact secrets and PII; use structured logging with filters; segregate debug vs prod logging.
- **Hardcoded secrets → Externalize and rotate**
Store secrets in a vault/secret store; inject at runtime; use short-lived credentials and rotation.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.
- **Sensitive data in logs → Implement secure logging**
Mask/redact secrets and PII; use structured logging with filters; segregate debug vs prod logging.
- **Hardcoded secrets → Externalize and rotate**
Store secrets in a vault/secret store; inject at runtime; use short-lived credentials and rotation.
- **Missing authZ/authN in code → Enforce checks at every layer**
Verify identity (authN) and permissions (authZ) per request; apply RBAC/ABAC; don't rely solely on gateways.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.
- **Sensitive data in logs → Implement secure logging**
Mask/redact secrets and PII; use structured logging with filters; segregate debug vs prod logging.
- **Hardcoded secrets → Externalize and rotate**
Store secrets in a vault/secret store; inject at runtime; use short-lived credentials and rotation.
- **Missing authZ/authN in code → Enforce checks at every layer**
Verify identity (authN) and permissions (authZ) per request; apply RBAC/ABAC; don't rely solely on gateways.
- **Unvetted/outdated libraries → Govern dependencies**
Pin versions, maintain SBOM, run SCA in CI, patch regularly, and prefer well-maintained libraries.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.
- **Sensitive data in logs → Implement secure logging**
Mask/redact secrets and PII; use structured logging with filters; segregate debug vs prod logging.
- **Hardcoded secrets → Externalize and rotate**
Store secrets in a vault/secret store; inject at runtime; use short-lived credentials and rotation.
- **Missing authZ/authN in code → Enforce checks at every layer**
Verify identity (authN) and permissions (authZ) per request; apply RBAC/ABAC; don't rely solely on gateways.
- **Unvetted/outdated libraries → Govern dependencies**
Pin versions, maintain SBOM, run SCA in CI, patch regularly, and prefer well-maintained libraries.
- **Leaky error handling → Standardize error responses**
Return generic client errors; log detailed stack traces internally; map exceptions to safe messages.

1. Secure Coding Practices (what we should do)

- **Untrusted input handling → Enforce strict validation & normalization**
Validate at boundaries (API layer), use allow-lists, constrain types/lengths/formats, and reject early.
- **Dynamic query/command construction → Use parameterization & safe APIs**
Prepared statements/ORM bind variables; avoid shell execution or use vetted libraries with escaping.
- **Overexposed API responses → Apply least-data principle**
Return DTOs/view models; explicitly map fields; exclude sensitive attributes by default.
- **Happy-path-only logic → Add defensive programming**
Validate invariants, handle edge cases, use guard clauses, and fail securely on invalid states.
- **Sensitive data in logs → Implement secure logging**
Mask/redact secrets and PII; use structured logging with filters; segregate debug vs prod logging.
- **Hardcoded secrets → Externalize and rotate**
Store secrets in a vault/secret store; inject at runtime; use short-lived credentials and rotation.
- **Missing authZ/authN in code → Enforce checks at every layer**
Verify identity (authN) and permissions (authZ) per request; apply RBAC/ABAC; don't rely solely on gateways.
- **Unvetted/outdated libraries → Govern dependencies**
Pin versions, maintain SBOM, run SCA in CI, patch regularly, and prefer well-maintained libraries.
- **Leaky error handling → Standardize error responses**
Return generic client errors; log detailed stack traces internally; map exceptions to safe messages.
- **"Internal is trusted" assumption → Treat all inputs as untrusted**
Apply the same validation, auth, and rate limits to internal APIs; enforce service identity where possible.

Secure Coding Practice - Code Review Checklist

1. Input Validation & Data Handling

- All external inputs (body, query, headers, files) are validated at entry points
- Allow-list validation used where applicable (not just deny-list)
- Length, format, type, and range are enforced
- Invalid input is rejected early with safe error messages
- File paths, URLs, and identifiers are sanitized (no path traversal risk)

Secure Coding Practice - Code Review Checklist

1. Input Validation & Data Handling

- All external inputs (body, query, headers, files) are validated at entry points
- Allow-list validation used where applicable (not just deny-list)
- Length, format, type, and range are enforced
- Invalid input is rejected early with safe error messages
- File paths, URLs, and identifiers are sanitized (no path traversal risk)

2. Authentication & Authorization

- All protected endpoints require authentication
- Authorization checks are enforced per action (not just login)
- Access is scoped to the current user/tenant only
- Admin/internal endpoints are explicitly secured
- Backend enforces auth (not relying solely on gateway/frontend)

Secure Coding Practice - Code Review Checklist

3. Data Access & Injection Prevention

- Parameterized queries / ORM bindings are used (no string concatenation)
- No dynamic execution of SQL, shell, or system commands
- Only required fields are queried and returned
- Sensitive records are filtered (status, ownership, tenant)
- Pagination and limits applied to list endpoints

Secure Coding Practice - Code Review Checklist

3. Data Access & Injection Prevention

- Parameterized queries / ORM bindings are used (no string concatenation)
- No dynamic execution of SQL, shell, or system commands
- Only required fields are queried and returned
- Sensitive records are filtered (status, ownership, tenant)
- Pagination and limits applied to list endpoints

4. API Response Safety

- DTOs/ViewModels used (no direct entity exposure)
- No sensitive data in responses (passwords, tokens, secrets, internal flags)
- Error responses are generic; no stack traces exposed
- HTTP status codes are consistent and meaningful
- Data exposure follows least-privilege principle

Secure Coding Practice - Code Review Checklist

5. Secrets & Configuration Management

- No hardcoded credentials, API keys, or tokens in code
- No secrets committed to repositories or config files
- Secrets are injected securely at runtime (env/secret manager)
- Default or shared credentials are not used
- Environment separation is enforced (dev/staging/prod)

Secure Coding Practice - Code Review Checklist

5. Secrets & Configuration Management

- No hardcoded credentials, API keys, or tokens in code
- No secrets committed to repositories or config files
- Secrets are injected securely at runtime (env/secret manager)
- Default or shared credentials are not used
- Environment separation is enforced (dev/staging/prod)

6. Logging & Monitoring

- Logs do not contain sensitive data (passwords, tokens, PII(Personal Identifiable Information))
- Sensitive values are masked/redacted
- Security-relevant events are logged (auth failures, critical actions)
- Logs provide traceability without leaking internals
- Debug-level logs are restricted in production

Secure Coding Practice - Code Review Checklist

7. Error Handling

- Exceptions are handled centrally and consistently
- User-facing errors are safe and non-revealing
- Internal logs capture detailed diagnostic info
- No raw exception messages returned to clients
- Failure paths do not bypass security checks

Secure Coding Practice - Code Review Checklist

7. Error Handling

- Exceptions are handled centrally and consistently
- User-facing errors are safe and non-revealing
- Internal logs capture detailed diagnostic info
- No raw exception messages returned to clients
- Failure paths do not bypass security checks

8. Business Logic Integrity

- Critical values (price, balance, roles) are computed server-side
- Client input is not trusted for sensitive decisions
- State transitions are validated (no illegal transitions)
- Duplicate/retry requests handled safely (idempotency where needed)
- Authorization enforced within business workflows

Secure Coding Practice - Code Review Checklist

9. File Handling (Upload/Download)

- File type, size, and extension validated
- MIME type not solely trusted
- Files stored with safe naming (no user-controlled paths)
- No execution of uploaded files
- Download endpoints enforce authorization

Secure Coding Practice - Code Review Checklist

9. File Handling (Upload/Download)

- File type, size, and extension validated
- MIME type not solely trusted
- Files stored with safe naming (no user-controlled paths)
- No execution of uploaded files
- Download endpoints enforce authorization

10. Cryptography

- Passwords hashed with strong algorithms (e.g., bcrypt/argon2)
- No weak hashing (MD5/SHA1) for sensitive data
- Approved crypto libraries used (no custom crypto)
- Secure random generators used for tokens/IDs
- Encryption applied where required (data at rest/in transit) with latest strong algorithm

Secure Coding Practice - Code Review Checklist

11. Session & Token Management

- Tokens have expiration and proper scope
- Refresh tokens are protected and revocable
- Cookies use `HttpOnly`, `Secure`, `SameSite` appropriately
- Tokens are not logged or exposed unnecessarily
- Session termination/logout handled correctly

Secure Coding Practice - Code Review Checklist

11. Session & Token Management

- Tokens have expiration and proper scope
- Refresh tokens are protected and revocable
- Cookies use `HttpOnly`, `Secure`, `SameSite` appropriately
- Tokens are not logged or exposed unnecessarily
- Session termination/logout handled correctly

12. Dependency & Library Governance

- Dependencies are necessary and justified
- Versions are pinned (no floating versions)
- Known vulnerable libraries are not used
- Libraries are actively maintained
- Unused dependencies are removed

Secure Coding Practice - Code Review Checklist

13. DoS & Resource Abuse Prevention

- Pagination/limits enforced on all list endpoints
- File upload size limits defined and enforced
- No unbounded loops over user-controlled input
- Expensive operations are protected (auth, rate limit, or async)

Secure Coding Practice - Code Review Checklist

13. DoS & Resource Abuse Prevention

- Pagination/limits enforced on all list endpoints
- File upload size limits defined and enforced
- No unbounded loops over user-controlled input
- Expensive operations are protected (auth, rate limit, or async)

14. Query & Data Efficiency

- No N+1 query patterns
- Queries optimized (indexes, selective fields)
- Query execution time is bounded (timeouts where applicable)

Secure Coding Practice - Code Review Checklist

13. DoS & Resource Abuse Prevention

- Pagination/limits enforced on all list endpoints
- File upload size limits defined and enforced
- No unbounded loops over user-controlled input
- Expensive operations are protected (auth, rate limit, or async)

14. Query & Data Efficiency

- No N+1 query patterns
- Queries optimized (indexes, selective fields)
- Query execution time is bounded (timeouts where applicable)

15. Rate Limiting & Throttling

- Sensitive endpoints (login, OTP, payment) are rate limited
- Retry logic uses backoff (no infinite retries)

Secure Coding Practice - Code Review Checklist

16. Memory & Processing Safety

- Large data handled via streaming (not full memory load)
- Deserialization is safe and controlled
- No excessive object creation from user input

Secure Coding Practice - Code Review Checklist

16. Memory & Processing Safety

- Large data handled via streaming (not full memory load)
- Deserialization is safe and controlled
- No excessive object creation from user input

17. Availability & Resilience

- External calls have timeouts
- Circuit breaker/fallback strategy exists where needed
- Heavy endpoints are authenticated and/or cached

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?
6. Can this be abused to exhaust system resources?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?
6. Can this be abused to exhaust system resources?
7. Does this fail securely under error conditions?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?
6. Can this be abused to exhaust system resources?
7. Does this fail securely under error conditions?
8. Unit Test coverage?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?
6. Can this be abused to exhaust system resources?
7. Does this fail securely under error conditions?
8. Unit Test coverage?
9. Integration Test applicable?

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?
6. Can this be abused to exhaust system resources?
7. Does this fail securely under error conditions?
8. Unit Test coverage?
9. Integration Test applicable?
10. What's the result of Linter

Reviewer Questions (Must Ask Before Approval)

1. Business test cases defined?
2. What inputs can an attacker control here?
3. Can this expose sensitive data (directly or indirectly)?
4. Can one user access another user's data?
5. What happens under malformed, repeated, or extreme input?
6. Can this be abused to exhaust system resources?
7. Does this fail securely under error conditions?
8. Unit Test coverage?
9. Integration Test applicable?
10. What's the result of Linter
11. What's the result of local SAST (e.g semgrep scan)

SAST (Static Application Security Testing)

What it is:

SAST analyzes your **source code (or bytecode/binaries)** without running the app to find security flaws early - typically during development or in CI.

What it does (at code level):

- Detects **injection risks** (SQL/OS/LDAP) from unsafe data flows
- Flags **hardcoded secrets** (API keys, passwords, tokens)
- Finds **insecure APIs/patterns** (weak crypto, unsafe deserialization)
- Identifies **missing validations** on untrusted input
- Highlights **auth/authZ gaps** (e.g., unprotected endpoints in certain frameworks)
- Provides **dataflow/taint analysis**: tracks user input → sensitive sinks

Output:

Issues with file/line, severity, and remediation guidance.

Limitations:

May produce **false positives**

SAST (Static Application Security Testing)

1) Choose a tool

- **Open-source:** SonarQube, Semgrep
- **Commercial/advanced:** Checkmarx, Fortify Static Code Analyzer
- **Git-native:** GitHub CodeQL

2) Run it locally (developer loop)

- Add a pre-commit or local command (example: `semgrep scan`)
- Fix **high/critical** findings before pushing

3) Integrate into CI/CD (mandatory gate)

- Run SAST on every PR/merge
- **Fail the build** on:
 - High/Critical issues
 - New vulnerabilities (baseline old ones if needed)

SCA (Software Composition Analysis)

What it is:

SCA analyzes your **third-party dependencies** (libraries, packages, containers) to identify **known vulnerabilities, license risks, and outdated components**.

What it does:

- Detects **known CVEs**(common vulnerabilities & exposures) in dependencies
- Flags **outdated libraries** and suggests safe versions
- Checks **license compliance** (e.g., GPL vs permissive)
- Monitors for **newly disclosed vulnerabilities** after release

Output:

List of vulnerable components, severity, affected versions, and upgrade paths.

Limitations:

Cannot find unknown (zero-day) issues

SCA (Software Composition Analysis)

1) Choose a tool

- **Open-source:** OWASP Dependency-Check, Snyk (free tier)
- **Git-native:** GitHub Dependabot
- **Commercial:** Black Duck

2) Integrate into CI/CD (must-have)

- Run on every PR/merge
- **Fail build if:**
 - High/Critical vulnerabilities found
 - Disallowed licenses detected

DAST (Dynamic Application Security Testing)

What it is:

DAST tests your **running application** (like a real attacker would) by sending crafted requests and analyzing responses—**no access to source code required**.

What it does (at runtime):

- Finds **injection vulnerabilities** (SQL/XSS/command) via payloads
- Detects **broken authentication & session issues**
- Identifies **misconfigurations** (headers, CORS, cookies)
- Discovers **exposed endpoints** and weak access control
- Checks **input handling in real execution paths**
- Validates **HTTP security headers** (CSP, HSTS, etc.)

Output:

Exploitable issues with URLs/endpoints, payloads used, and evidence.

Limitations:

Might raise False Positive

DAST (Dynamic Application Security Testing)

1) Choose a tool

- **Open-source:** OWASP ZAP
- **Commercial:** Burp Suite (Pro/Enterprise)

2) Run against a deployed environment

- Use **staging / test environment** (never first-time on production)
- Ensure realistic data and configs (auth flows enabled)

Container Image Scanning

What it is:

Analyzes your **built container images** (e.g., Docker images) to find known vulnerabilities, misconfigurations, and risky components before deployment.

What it does:

- Detects **OS package CVE**(Common Vulnerabilities & Exposures) (e.g., in `openssl`, `glibc`, `bash`)
- Flags **vulnerable application dependencies** baked into the image
- Identifies **misconfigurations** (running as root, weak permissions)
- Surfaces **secrets accidentally included** in layers
- Highlights **outdated base images**

Output:

Vulnerabilities by layer/package, severity, and remediation (upgrade, patch, or rebuild).

Container Image Scanning

1) Choose a scanner

- **Open-source:** Trivy, Clair
- **Commercial/registry-integrated:** Aqua Security, Anchore

2) Scan locally (developer loop)

- `trivy image my-app:latest`
- Fix **High/Critical** issues before pushing

3) Scan in registry (continuous)

- Enable scanning in your container registry (on push)
- Get alerts when new CVEs affect existing images

Kubernetes Security

What it is:

hardening the cluster, workloads, and access controls so that a compromise in one place doesn't become a **cluster-wide breach**.

What it protects:

- **Cluster control plane** (API server, etcd)
- **Workloads** (Pods, containers)
- **Access** (users, service accounts)
- **Network communication** between services
- **Secrets and configuration data**

What it focuses on:

- **Who can do what** → Access control (RBAC)
- **How containers run** → SecurityContext (privileges, user, capabilities)
- **Who can talk to whom** → Network isolation
- **What gets deployed** → Admission control & policies
- **Where secrets live** → Secure storage and access

Kubernetes Security

What teams usually do (problem)

- Use default service accounts everywhere

Kubernetes Security

What teams usually do (problem)

- Use **default service accounts everywhere**
- Run containers as **root / privileged mode**

Kubernetes Security

What teams usually do (problem)

- Use **default service accounts everywhere**
- Run containers as **root / privileged mode**
- No **RBAC restrictions** (over-permissive roles)

Kubernetes Security

What teams usually do (problem)

- Use **default service accounts everywhere**
- Run containers as **root / privileged mode**
- No **RBAC restrictions** (over-permissive roles)
- No **network policies** (flat network)

Kubernetes Security

What teams usually do (problem)

- Use **default service accounts everywhere**
- Run containers as **root / privileged mode**
- No **RBAC restrictions** (over-permissive roles)
- No **network policies** (flat network)
- Store **secrets in plain YAML**

Kubernetes Security

What teams usually do (problem)

- Use **default service accounts everywhere**
- Run containers as **root / privileged mode**
- No **RBAC restrictions** (over-permissive roles)
- No **network policies** (flat network)
- Store **secrets in plain YAML**
- Expose **dashboard/API publicly**

Kubernetes Security

1) Enforce RBAC (least privilege)

- Create **role-based permissions** (not cluster-admin for all)
- Assign **specific roles to specific services/users**

Kubernetes Security

1) Enforce RBAC (least privilege)

- Create **role-based permissions** (not cluster-admin for all)
- Assign **specific roles to specific services/users**

2) Secure Pods (SecurityContext)

- Run containers as **non-root user**
- Disable privilege escalation
- Drop unnecessary Linux capabilities

Kubernetes Security

1) Enforce RBAC (least privilege)

- Create **role-based permissions** (not cluster-admin for all)
- Assign **specific roles to specific services/users**

2) Secure Pods (SecurityContext)

- Run containers as **non-root user**
- Disable privilege escalation
- Drop unnecessary Linux capabilities

3) Use Network Policies

- Restrict communication between services
- Allow only required traffic (e.g., API → DB)

Kubernetes Security

1) Enforce RBAC (least privilege)

- Create **role-based permissions** (not cluster-admin for all)
- Assign **specific roles to specific services/users**

2) Secure Pods (SecurityContext)

- Run containers as **non-root user**
- Disable privilege escalation
- Drop unnecessary Linux capabilities

3) Use Network Policies

- Restrict communication between services
- Allow only required traffic (e.g., API → DB)

4) Protect Secrets

- Store secrets in **Kubernetes Secrets (encoded)** or external vault
- Do not hardcode in manifests
- Restrict access via RBAC

Kubernetes Security

5) Enable Admission Controls / Policies

- Enforce rules like:
 - No root containers
 - Only approved images
- Use tools like Open Policy Agent (OPA/Gatekeeper)

Kubernetes Security

5) Enable Admission Controls / Policies

- Enforce rules like:
 - No root containers
 - Only approved images
- Use tools like Open Policy Agent (OPA/Gatekeeper)

6) Scan and verify images before deployment

- Only deploy **trusted, scanned images**
- Block vulnerable images at admission stage

Kubernetes Security

5) Enable Admission Controls / Policies

- Enforce rules like:
 - No root containers
 - Only approved images
- Use tools like Open Policy Agent (OPA/Gatekeeper)

6) Scan and verify images before deployment

- Only deploy **trusted, scanned images**
- Block vulnerable images at admission stage

7) Secure API Server & Access

- Disable anonymous access
- Use strong authentication (certs, tokens, IAM)
- Restrict API exposure (no public access if not needed)

Kubernetes Security

5) Enable Admission Controls / Policies

- Enforce rules like:
 - No root containers
 - Only approved images
- Use tools like Open Policy Agent (OPA/Gatekeeper)

6) Scan and verify images before deployment

- Only deploy **trusted, scanned images**
- Block vulnerable images at admission stage

7) Secure API Server & Access

- Disable anonymous access
- Use strong authentication (certs, tokens, IAM)
- Restrict API exposure (no public access if not needed)

8) Monitor & Audit

- Enable audit logs
- Monitor suspicious activity
- Use runtime security tools if possible

Zero Trust & Network Policies

What it is:

Zero Trust = *never trust by location (internal/external), always verify identity and intent.*

Network Policies = enforce **who can talk to whom**, at **network level**, based on least privilege.

Where Teams Usually Fail:

- Trust internal IPs instead of identity
- No segmentation (VM or K8s)
- Expose admin/internal endpoints inside network
- No service-to-service authentication
- Over-permissive firewall rules
- App + DB on same network
- Open ports broadly (e.g., 0.0.0.0)
- No firewall rules between services
- Internal APIs have weak/no auth

Typical assumption: “Internal network is safe”

Reality:

- One compromised laptop / server / pod → **full lateral movement**
- Flat networks = attacker paradise

Zero Trust & Network Policies

Core Zero Trust Principles

- **Verify explicitly**
Every request must be authenticated & authorized

Zero Trust & Network Policies

Core Zero Trust Principles

- **Verify explicitly**
Every request must be authenticated & authorized
- **Least privilege access**
Minimum access required, nothing more

Zero Trust & Network Policies

Core Zero Trust Principles

- **Verify explicitly**
Every request must be authenticated & authorized
- **Least privilege access**
Minimum access required, nothing more
- **Assume breach**
Design as if attacker is already inside

Zero Trust & Network Policies

For VM / Monolith

- Enforce firewall rules (deny by default)
- Separate network zones (App vs DB)
- Add authentication even for internal APIs
- Use VPN or restricted access for admin endpoints

Zero Trust & Network Policies

For VM / Monolith

- Enforce firewall rules (deny by default)
- Separate network zones (App vs DB)
- Add authentication even for internal APIs
- Use VPN or restricted access for admin endpoints

For Kubernetes

- Apply **default deny-all NetworkPolicy**
- Allow only required service communication
- Use service identity (mTLS if advanced)
- Restrict ingress/egress explicitly

Zero Trust & Network Policies

For VM / Monolith

- Enforce firewall rules (deny by default)
- Separate network zones (App vs DB)
- Add authentication even for internal APIs
- Use VPN or restricted access for admin endpoints

For Kubernetes

- Apply **default deny-all NetworkPolicy**
- Allow only required service communication
- Use service identity (mTLS if advanced)
- Restrict ingress/egress explicitly

Tools (optional mention)

- Istio (mTLS, identity-based access)
- Calico (network enforcement)

Secrets Management

What it is:

The practice of **securely** storing, accessing, rotating, and auditing **sensitive data** such as passwords, API keys, tokens, and private keys - **separate from code**.

What counts as a “secret”:

- Database credentials
- API keys / third-party tokens
- JWT signing keys
- Encryption keys / certificates
- SSH keys, service account tokens

Secrets Management

What it is:

The practice of **securely** storing, accessing, rotating, and auditing **sensitive data** such as passwords, API keys, tokens, and private keys - **separate from code**.

What counts as a “secret”:

- Database credentials
- API keys / third-party tokens
- JWT signing keys
- Encryption keys / certificates
- SSH keys, service account tokens

What teams usually do (problem):

- Hardcode secrets in code or config files
- Commit secrets to Git (even once = permanent risk)
- Reuse same credentials across environments
- No rotation (“set and forget”)
- Share secrets over chat/email

Secrets Management

What proper Secrets Management does

- **Centralizes storage** (not scattered in code/repos)
- **Controls access** (who/what can read which secret)
- **Injects secrets at runtime** (not baked into code/image)
- **Rotates secrets regularly**
- **Audits access and usage**

Secrets Management

What proper Secrets Management does

- **Centralizes storage** (not scattered in code/repos)
- **Controls access** (who/what can read which secret)
- **Injects secrets at runtime** (not baked into code/image)
- **Rotates secrets regularly**
- **Audits access and usage**

Use a secret store

- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault

Secrets Management

What proper Secrets Management does

- **Centralizes storage** (not scattered in code/repos)
- **Controls access** (who/what can read which secret)
- **Injects secrets at runtime** (not baked into code/image)
- **Rotates secrets regularly**
- **Audits access and usage**

Use a secret store

- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault

Apply least privilege

- Each service gets **only the secrets it needs**
- No shared/global credentials

Thank you